MODULE INTERACTIONS

Part—17

This article, which is part of the series on Linux device drivers, demonstrates various interactions with a Linux module.

As Shweta and Pugs gear up for their final semester's project on Linux drivers, they're closing in on some final titbits of technical romancing. This mainly includes the various communications with a Linux module (dynamically loadable and unloadable driver) like accessing its variables, calling its functions, and passing parameters to it.

Global variables and functions

One might wonder what the big deal is about accessing a module's variables and functions from outside it. Just make them global, declare them *extern* in a header, include the header and access, right? In the general application development paradigm, it's this simple—but in kernel development, it isn't inspite of recommendations to make everything static, by default there always have been cases where non-static globals may be needed. A simple example could be a driver spanning multiple files, with function(s) from one file needing to be called in the other. Now, to avoid any kernel name-space collisions even with such cases, every module is embodied in its own name-space. And we know that two modules with the same name cannot be loaded at the same time. Thus, by default, zero collision is achieved. However, this also implies that, by default, nothing from a module can be made really global throughout the kernel, even if we want to. And so, for exactly such scenarios, the *<linux/module.h>* header defines the following macros:

- *EXPORT_SYMBOL(sym)*
- *EXPORT_SYMBOL_GPL(sym)*
- *EXPORT_SYMBOL_GPL_FUTURE(sym)*

Each of these exports the symbol passed as their parameter, additionally putting them in one of the *default, _gpl or _gpl_future* sections, respectively. Hence, only one of them needs to be used for a particular symbol—though the symbol could be either a variable name or a function name. Here's the complete code (*our_glob_syms.c*) to demonstrate this:

```
#include <linux/module.h>
#include <linux/device.h>

static struct class *cool_cl;
static struct class *get_cool_cl(void)
{
    return cool_cl;
}
EXPORT_SYMBOL(cool_cl);
EXPORT_SYMBOL_GPL(get_cool_cl);

static int __init glob_sym_init(void)
{
    if (IS_ERR(cool_cl = class_create(THIS_MODULE, "cool")))
    /* Creates /sys/class/cool/ */
    {
        return PTR_ERR(cool_cl);
    }
    return 0;
}

static void __exit glob_sym_exit(void)
{
    /* Removes /sys/class/cool/ */
    class_destroy(cool_cl);
}

module_init(glob_sym_init);
module_exit(glob_sym_exit);

MODULE_LICENSE("GPL");
MODULE_AUTHOR("Anil Kumar Pugalia <email@sarika-pugs.com>");
MODULE_DESCRIPTION("Global Symbols exporting Driver");
```

Each exported symbol also has a corresponding structure placed into (each of) the kernel symbol table